



**VIDYASHILP
UNIVERSITY**

ASSIGNMENT I REPORT
Advanced Topics in Deep Learning

**KNOWLEDGE DISTILLATION WITH TERNARY- WEIGHT
QUANTIZATION- AWARE TRAINING**

ResNet34 (FP32 Teacher) and ResNet18 (Ternary Student) on CIFAR- 10

Submitted on: 16/09/2026

Submitted by: **Raju**

Student ID: **2023UG000147**

Contents

1. Problem Statement	3
2. Data Pipeline	3
3. Methodology	3
3.1 Stage 1 — Teacher Model (ResNet34)	3
3.2 Stage 2 — Baseline Student Model (ResNet18 FP32)	4
3.3 Stage 3 — Ternary Weight Quantization & Knowledge Distillation	4
3.4 Stage 4 — Joint KD + QAT Student Model	5
3.5 Stage 5 — Ablation Study on Temperature	7
4. Results on Unseen Data	8
5. Discussion and Limitations	9
6. Reproducibility	9
7. Conclusion	9
8. References	10

1. Problem Statement

The goal of this assignment is to build a small, fast image classifier that is almost as accurate as a big one. We do this in three steps. First, we train a large “teacher” network (ResNet34) on CIFAR- 10 until it is accurate. Second, we build a smaller “student” network (ResNet18) whose weights are restricted to only three values: $-\alpha$, 0, or $+\alpha$ (this is called ternary quantization). Third, we train the student using Knowledge Distillation (KD), so it learns from the teacher's predictions, not only from the true labels. This is called QAT (Quantization- Aware Training) because the student sees the ternary weights during training itself, not after.

We compare three models at the end: the FP32 teacher, an FP32 ResNet18 baseline trained without KD, and the ternary ResNet18 student trained with KD + QAT. We check accuracy, model size, and how much the model shrinks.

2. Data Pipeline

CIFAR- 10 has 60,000 small colour images (32×32 pixels) in 10 classes, split into 50,000 training images and 10,000 test images. In this project:

- 90% of the training images are used for training, and 10% are kept aside as a validation set (used only to pick the best checkpoint).
- The test set is used only once at the end, so it never influences training or model selection.
- Training images are randomly cropped and flipped to make the model more robust. Validation and test images are not changed.
- All images are normalised using the standard CIFAR- 10 mean and standard deviation.
- Batch size is 128. A fixed random seed (42) is used everywhere so results can be repeated.

3. Methodology

This project has five stages, described below. Each stage builds on the checkpoint saved by the previous one.

3.1 Stage 1 - Teacher Model (ResNet34)

The teacher is a ResNet34 network, adapted for CIFAR- 10's small 32×32 images (a smaller first layer is used, with no aggressive downsampling, so that important detail is not lost early on). It is trained normally, in full precision (FP32), using the following settings:

Setting	Value
Parameters	21.28 million
Optimiser	SGD, momentum 0.9
Learning- rate schedule	Cosine decay, starting at 0.1
Weight decay / Label smoothing	5×10^{-4} / 0.1
Epochs / Batch size / Seed	160 / 128 / 42

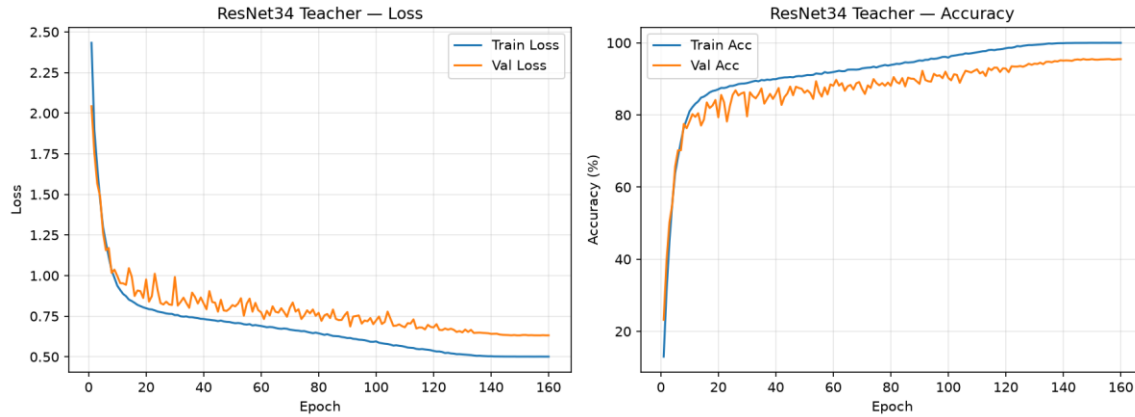


Figure 1. Teacher (ResNet34) training and validation loss/accuracy over 160 epochs.

Result: the teacher reached 95.50% validation accuracy and 94.98% test accuracy (best checkpoint at epoch 147). The gap between train accuracy (99.55%) and test accuracy is 4.57 percentage points, which is a normal, healthy amount of overfitting for this kind of training run - not a sign of a problem.

3.2 Stage 2 - Baseline Student Model (ResNet18 FP32)

Before adding KD or quantization, we train a plain FP32 ResNet18 with the same recipe as the teacher (same optimiser, schedule, epochs, seed), but with no help from the teacher. This gives us a fair “no- compression, no- KD” baseline to compare against later.

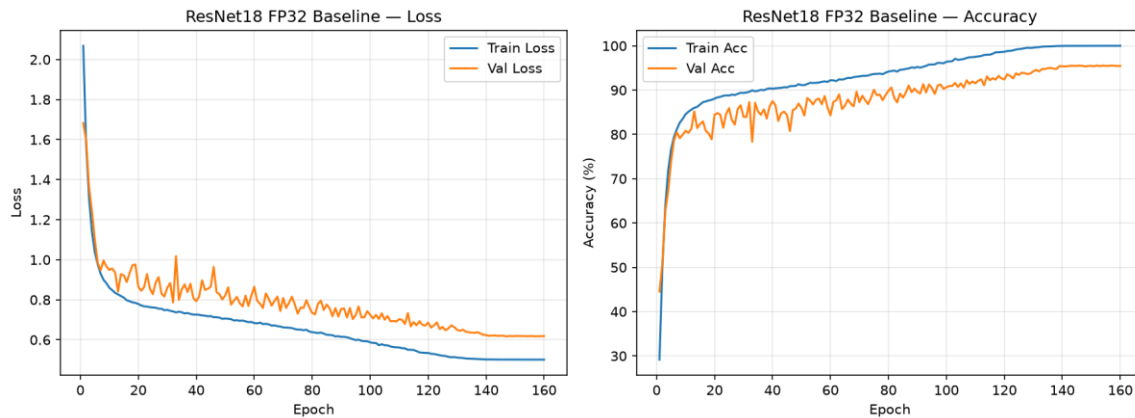


Figure 2. FP32 ResNet18 baseline training and validation loss/accuracy over 160 epochs.

Result: 95.56% validation accuracy, 95.08% test accuracy, 11.17 million parameters (best checkpoint at epoch 152). Interestingly, this smaller ResNet18 baseline scores slightly higher than the bigger ResNet34 teacher. This can happen on CIFAR- 10, since it is a fairly small, easy dataset where extra depth does not always help - we discuss this more in Section 5.

3.3 Stage 3 - Ternary Weight Quantization and Knowledge Distillation

This stage explains the two ideas combined in the student model: making weights ternary, and teaching the student using the teacher's soft predictions.

Ternary weights

A normal weight can be any decimal number. A ternary weight can only be one of three values: $-\alpha$, 0, or $+\alpha$, where α is a single positive number (a scale) shared by a group of weights. The network still keeps a full- precision “latent”

copy of every weight in the background — this is the copy the optimiser actually updates. On every forward pass, this latent weight is converted to its ternary version using a threshold Δ :

$$\Delta = 0.7 \times \text{average}(|W|) \quad \alpha = \text{average of } |W| \text{ for weights above } \Delta$$

Weights below the threshold Δ become 0; weights above it become $+\alpha$ or $-\alpha$ depending on their sign. This threshold rule ($\Delta = 0.7 \times \text{mean}(|W|)$) is the standard rule from the Ternary Weight Networks paper [3]. In this project, Δ and α are computed separately for each output channel (“per- channel”), which lets each filter pick the ternary values that suit it best.

Because this conversion is a step function, it has no useful gradient (it is flat almost everywhere, so normal backpropagation would get stuck). To fix this, a Straight- Through Estimator (STE) is used [5], [6]: the ternary weights are used going forward, but going backward, the gradient is passed straight through to the latent FP32 weight, as if no quantization happened. This lets the optimiser keep learning normally, while the forward pass still genuinely uses ternary weights.

Knowledge Distillation (KD)

Instead of only learning from the correct label, the student also learns from the teacher's full probability distribution over all 10 classes (the “soft targets”). A temperature T is used to soften the teacher's predictions, so the student can see which wrong classes the teacher considers “less wrong” - information a one- hot label alone cannot give. The combined loss is:

$$\text{Loss} = \lambda \times T^2 \times \text{KL}(\text{teacher}, \text{student}) + (1 - \lambda) \times \text{CrossEntropy}(\text{student}, \text{true label})$$

Here λ controls how much weight goes to the teacher's soft targets versus the true label, and T^2 rescales the soft-target loss so it stays balanced with the hard- label loss at different temperatures [2]. The teacher is frozen throughout (no gradients flow into it), so it acts as a fixed, stable source of guidance.

3.4 Stage 4 - Joint KD + QAT Student Model

The final student combines everything above: a ResNet18 with ternary weights inside every residual block (the very first and very last layers stay FP32, as is standard), trained with the combined KD + hard- label loss, while genuinely using ternary weights on every forward pass.

Setting	Value
Temperature T	4.0
KD weight λ	0.7
Ternary threshold scale	0.7 (per channel)
FP32 warm- up	First 5 epochs train normally, before ternary weights switch on
Epochs / Batch size / Seed	160 / 128 / 42

A short 5- epoch “warm- up” period trains the student normally (no quantization) before switching the ternary weights on. This gives the optimiser a slightly better starting point before the harder, quantized training begins. The training log shows validation accuracy actually dips during warm- up, then jumps up sharply the moment quantization switches on — this is expected and is explained in the caption below.

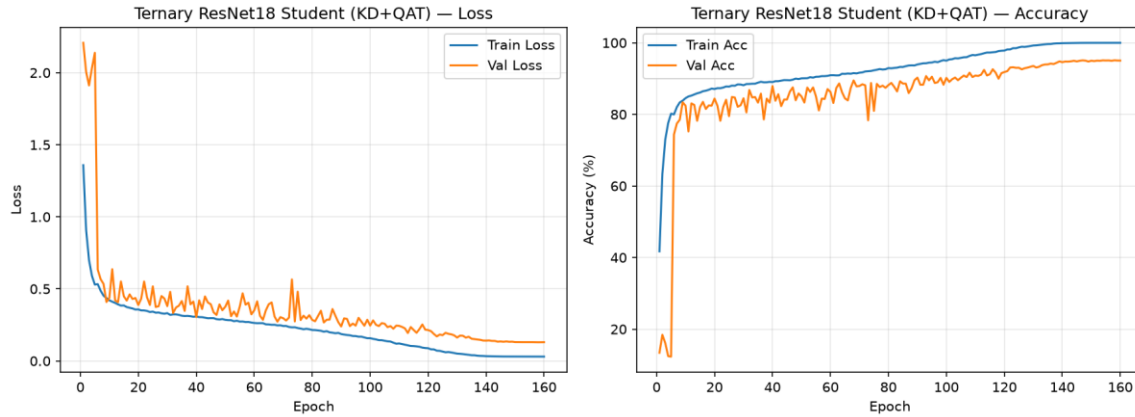


Figure 3. Ternary student (KD+QAT) training and validation loss/accuracy over 160 epochs. The jump around epoch 6 is where ternary weights switch on.

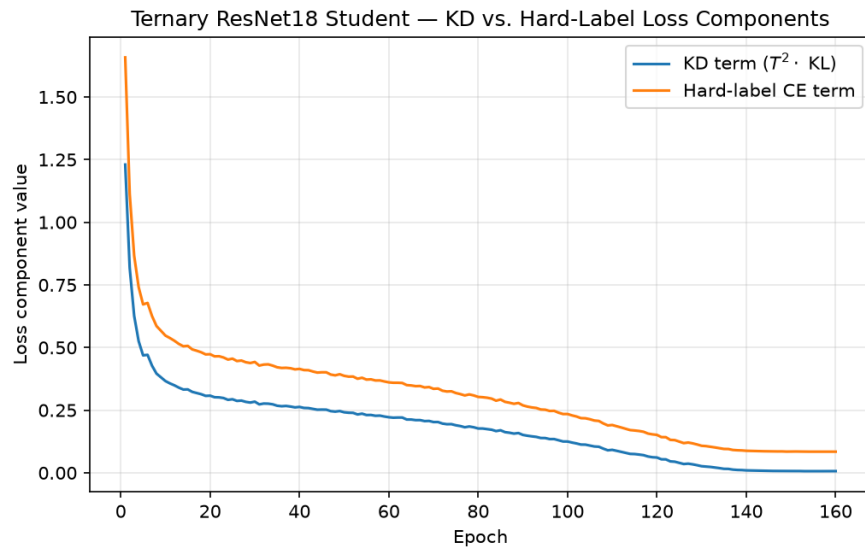


Figure 4. The two parts of the training loss: the KD (soft- target) term and the hard- label term. Both fall smoothly together.

Result: 95.10% validation accuracy, 94.63% test accuracy (best checkpoint at epoch 158). A checkpoint- verification check confirmed all 19 ternary layers in the saved model genuinely only contain the values $\{-\alpha, 0, +\alpha\}$ - the ternary constraint really is enforced, not just approximate.

Final comparison — all three models

Model	Params (M)	Test Acc %	Gap (pp)	Size (MB)	Sparsity %	Compression vs FP32 R18
ResNet34 Teacher (FP32)	21.28	94.98	4.57	81.19	0.00	0.53×
ResNet18 Baseline (FP32)	11.17	95.08	4.48	42.63	0.00	1.00×
ResNet18 Ternary (KD+QAT)	11.17	94.63	4.86	2.74	47.69	15.55×

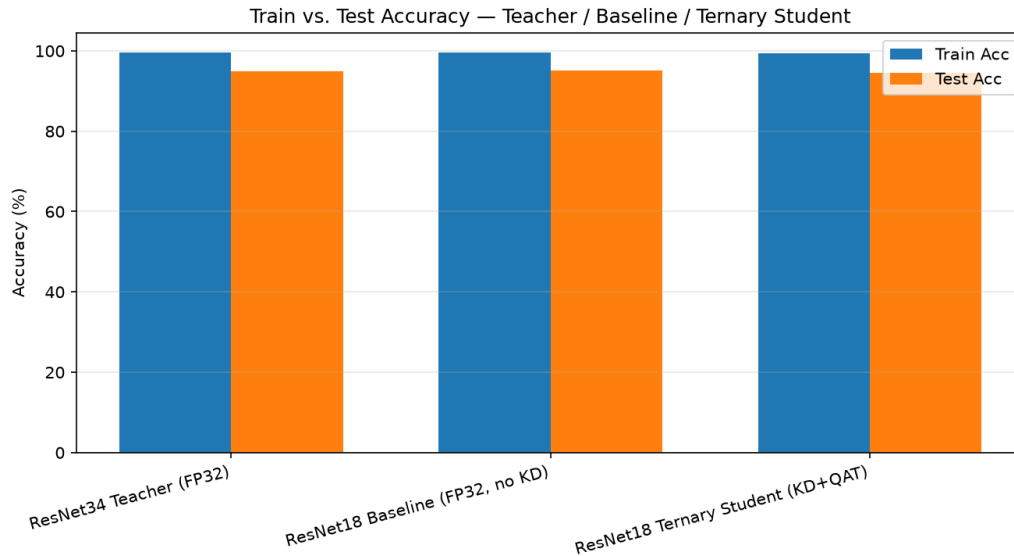


Figure 5. Train vs. test accuracy for all three models.

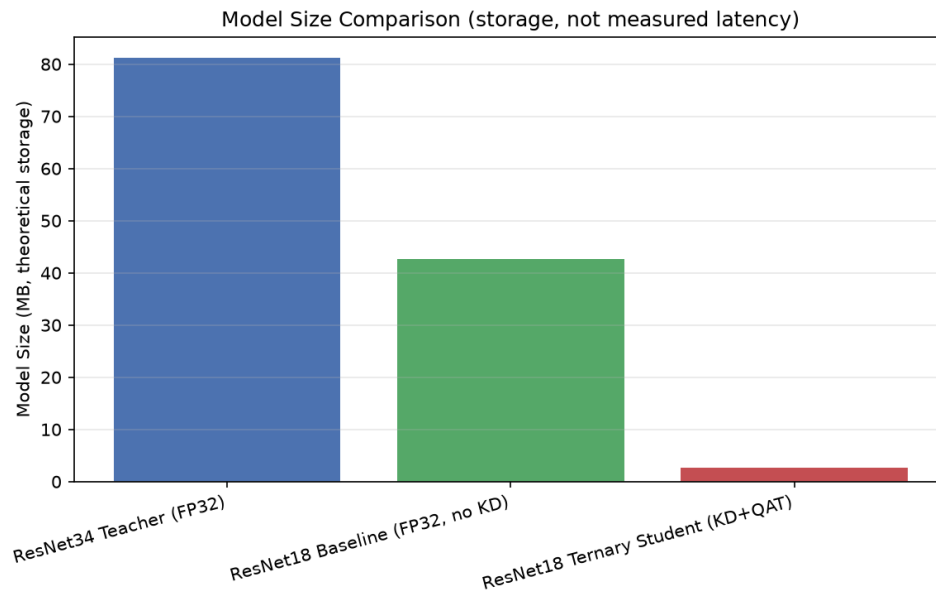


Figure 6. Theoretical storage size (MB) for all three models - a storage comparison, not a measured speed comparison (see Section 5).

The ternary student is only 0.45 percentage points behind the FP32 baseline in test accuracy, while using about 15.55× less storage. This shows that combining KD with QAT recovers almost all the accuracy that plain ternary quantization would otherwise lose.

3.5 Stage 5 - Ablation Study on Temperature

An ablation study changes one setting at a time to see its effect. Here, we vary the KD temperature $T \in \{1, 2, 4, 8\}$ while keeping λ fixed at 0.7 and everything else the same. Each setting is trained for 100 epochs (fewer than the full 160- epoch run in Section 3.4, to save time, since this is only a relative comparison).

Temperature T	Best Validation Accuracy (%)
1.0	94.88

Temperature T	Best Validation Accuracy (%)
2.0	94.80
4.0	94.92
8.0	94.72

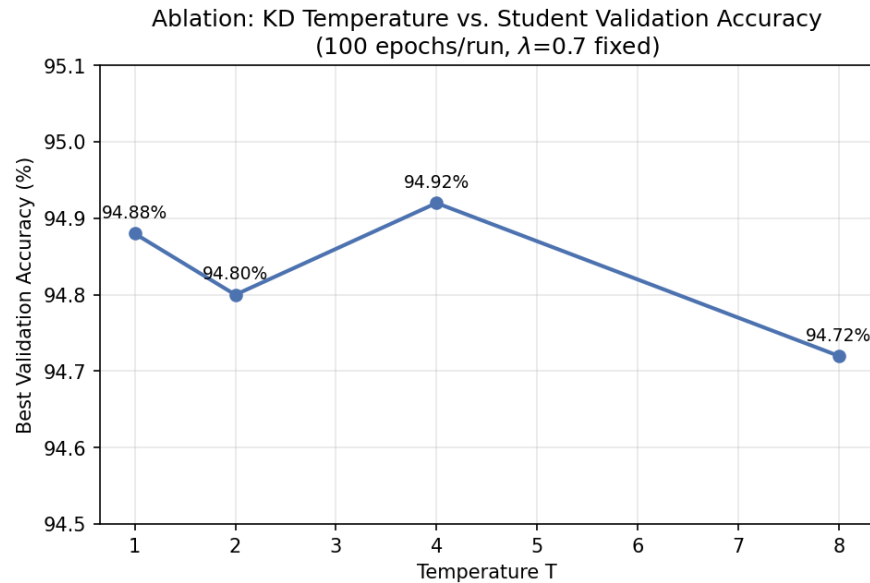


Figure 7. Best validation accuracy vs. KD temperature T (100 epochs per run, $\lambda = 0.7$ fixed).

All four results are close together, within a 0.20 percentage- point band. $T = 4$ is marginally the best, and $T = 8$ is marginally the worst, which roughly matches the usual idea that a middle temperature works best - but the difference is small enough that it should not be treated as a strong, certain result from a single run per setting. The main takeaway is that this model is not very sensitive to the exact KD temperature chosen, within the 1–8 range.

4. Results on Unseen Data

As a qualitative sanity check outside the requirements of the comparison table, the trained ternary student was also run on a local folder of 200 unlabeled images resized and normalised to match CIFAR- 10 preprocessing. Figure 8.1 shows the distribution of predicted classes and Figure 8.2 the distribution of prediction confidence (max softmax probability) across the 200 images; a sample grid of predictions is shown in Figure 8.3. Because these images are unlabeled, no accuracy figure is computed here - the check is purely qualitative, intended to confirm the model behaves sensibly on images from outside the CIFAR- 10 test split rather than to produce a reportable metric.

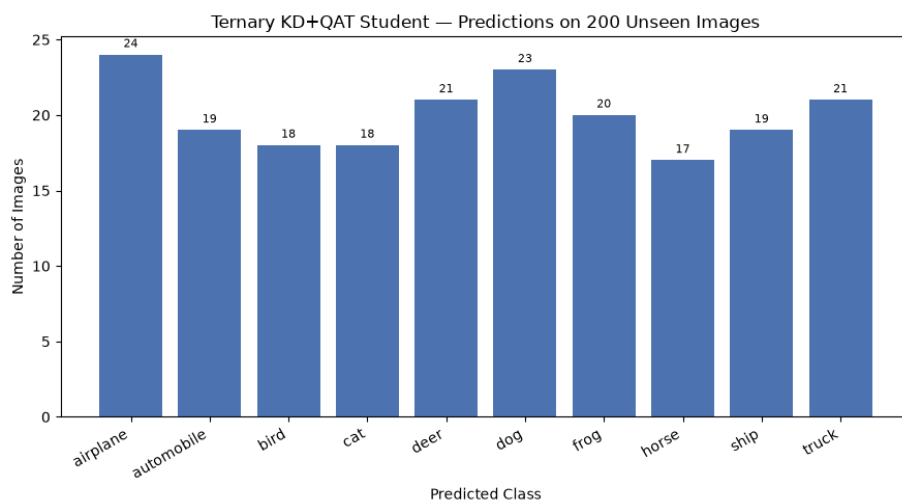


Figure 8.1 - Predicted- class distribution over 200 unseen images.

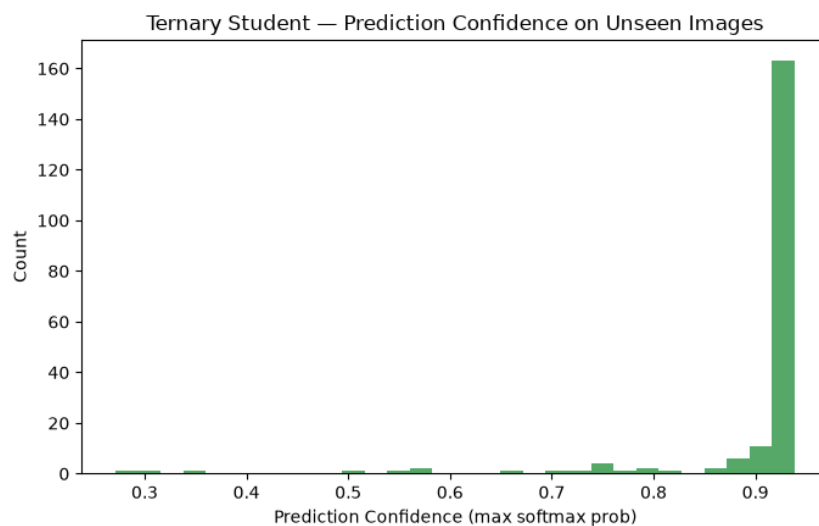


Figure 8.2 - Prediction- confidence histogram over 200 unseen images.

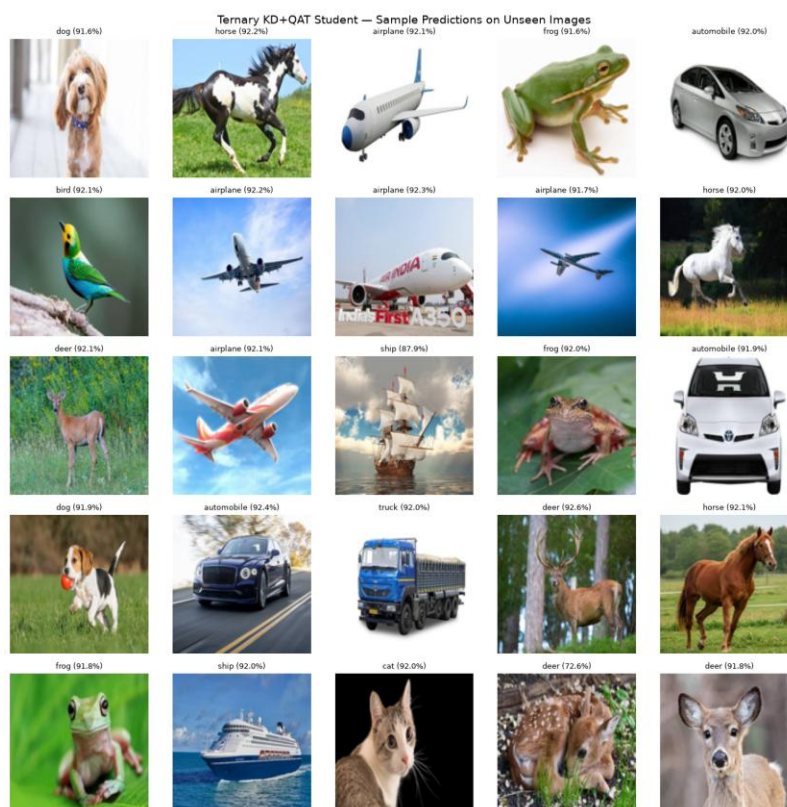


Figure 8.3 Sample predictions (label and confidence) on 25 of the 200 unseen images.

5. Discussion and Limitations

5.1 Storage compression is not the same as speed

The 15.55× compression number is a theoretical storage size, based on how many bits it would take to store ternary weights. It does not mean the model runs 15.55× faster. Normal GPUs do not have special hardware for 2-bit ternary math, so running this model as-is gives no real speed benefit - it may even be slightly slower, because of the extra work needed to turn ternary weights back into normal numbers before each layer runs. Getting a real speed-up would need special low-bit hardware or software kernels, which this project does not build or test.

5.2 The ablation ranking changed with more training

An earlier, shorter 30-epoch version of the temperature ablation gave a different ranking (T=1 best at 93.92%, T=4 worst at 93.38%, a 0.54-point spread) that seemed to favour low temperatures. The longer 100-epoch version in Section 3.5 gave a smaller spread (0.20 points) and a different ranking (T=4 best). This shows that a short ablation run can be misleading - the models may not have had enough time to properly separate from each other. The 100-epoch result is treated as the more reliable one here.

5.3 The baseline beating the teacher is a real, unusual result

The FP32 ResNet18 baseline (95.08% test accuracy) is slightly better than the bigger ResNet34 teacher (94.98%), even though both used the exact same training recipe. This is a genuine measured result, not a mistake. CIFAR-10 is a fairly easy dataset, and once a network family is already doing well on it, extra depth does not always add accuracy - sometimes it only adds capacity to overfit noise. This is worth knowing because it means the “teacher” guiding the student is not strictly the best possible model on this data.

5.4 All results are from a single run

Every number in this report (teacher, baseline, student, and each ablation setting) comes from one training run with one fixed random seed (42). No repeated runs or confidence intervals were computed. Small differences - like the 0.20- point spread in the temperature ablation, or the 0.45- point gap to the FP32 baseline - should be read as approximate, not statistically exact.

6. Reproducibility

- Random seed 42 is set at the start of every notebook, controlling the train/validation split and model initialisation.
- All settings (learning rate, epochs, temperature, λ , threshold scale, etc.) are stored in a single config file (src/config.py) and saved inside every checkpoint, so the exact settings used for any result can always be checked.
- Checkpoints are only saved when validation accuracy improves, so the “best epoch” numbers in this report are the real epoch selected, not just the last one.
- See the separate README.md file for full, step- by- step setup and run instructions for every notebook, in the correct order.

7. Conclusion

This project trained a strong ResNet34 teacher (94.98% test accuracy), an FP32 ResNet18 baseline (95.08%), and a ternary ResNet18 student trained with combined KD and QAT (94.63%). The ternary student loses only 0.45 percentage points of accuracy compared to the FP32 baseline, while needing about 15.55 $\times$ less storage. A temperature ablation showed the model is fairly insensitive to the exact KD temperature chosen. The main limitations are that storage compression does not automatically mean faster inference, and that all results come from single training runs rather than repeated, averaged runs. The next step for this project is to complete Section 4 by testing the final student model on genuinely unseen images.

8. References

- [1] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” in Proc. NIPS Deep Learning and Representation Learning Workshop, 2015.
- [2] F. Li, B. Zhang, and B. Liu, “Ternary weight networks,” arXiv:1605.04711, 2016.
- [3] C. Zhu, S. Han, H. Mao, and W. J. Dally, “Trained ternary quantization,” in Proc. Int. Conf. Learn. Represent. (ICLR), 2017.
- [4] Y. Bengio, N. Léonard, and A. Courville, “Estimating or propagating gradients through stochastic neurons for conditional computation,” arXiv:1308.3432, 2013.
- [5] A. Krizhevsky, “Learning multiple layers of features from tiny images,” Univ. of Toronto, Tech. Rep., 2009.